

BeadMPGNN — Full Forward Pass ($K = 1$)

Personal Reference

For one bead i with left neighbor L and right neighbor R .

1. Inputs

$$\mathbf{x}_{\text{node}} = [v_x, v_y, m, \text{is_fixed}] \in \mathbb{R}^4 \quad (1)$$

$$\mathbf{x}_{\text{edge}}^L = [\Delta x, \Delta y, \Delta v_x, \Delta v_y, m_L, \ell_0] \in \mathbb{R}^6 \quad (2)$$

$$\mathbf{x}_{\text{edge}}^R = [\Delta x, \Delta y, \Delta v_x, \Delta v_y, m_R, \ell_0] \in \mathbb{R}^6 \quad (3)$$

where Δ quantities are relative to the neighbor (e.g. $\Delta x = x_{\text{neighbor}} - x_{\text{self}}$).

2. Input Normalization

$$\hat{\mathbf{x}}_{\text{node}} = \frac{\mathbf{x}_{\text{node}} - \boldsymbol{\mu}_{\text{node}}}{\boldsymbol{\sigma}_{\text{node}}} \quad (4)$$

$$\hat{\mathbf{x}}_{\text{edge}} = \frac{\mathbf{x}_{\text{edge}} - \boldsymbol{\mu}_{\text{edge}}}{\boldsymbol{\sigma}_{\text{edge}}} \quad (5)$$

$\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are computed once from the training set and fixed during training (not learned).

3. ResMLPBlock (used by all encoders)

Given input $\mathbf{z}$ with input dim d_{in} and hidden dim $H = 64$:

$$\mathbf{z}' = W_{\text{proj}} \mathbf{z} + \mathbf{b}_{\text{proj}} \quad W_{\text{proj}} \in \mathbb{R}^{H \times d_{\text{in}}} \quad (6)$$

$$\hat{\mathbf{z}} = \text{LayerNorm}(\mathbf{z}') \quad (7)$$

$$\mathbf{a} = \text{GELU}(W_1 \hat{\mathbf{z}} + \mathbf{b}_1) \quad W_1 \in \mathbb{R}^{2H \times H} \quad (\text{expand}) \quad (8)$$

$$\mathbf{o} = W_2 \mathbf{a} + \mathbf{b}_2 \quad W_2 \in \mathbb{R}^{H \times 2H} \quad (\text{contract}) \quad (9)$$

$$\text{output} = \mathbf{z}' + \mathbf{o} \quad (\text{residual connection}) \quad (10)$$

4. Node Encoder

$$\mathbf{h}_{\text{self}} = \text{ResMLPBlock}(\hat{\mathbf{x}}_{\text{node}}) \quad \mathbb{R}^4 \rightarrow \mathbb{R}^{64} \quad (11)$$

5. Edge Encoder (shared weights for L and R)

$$\mathbf{h}_{\text{edge}}^L = \text{ResMLPBlock}(\hat{\mathbf{x}}_{\text{edge}}^L) \quad \mathbb{R}^6 \rightarrow \mathbb{R}^{64} \quad (12)$$

$$\mathbf{h}_{\text{edge}}^R = \text{ResMLPBlock}(\hat{\mathbf{x}}_{\text{edge}}^R) \quad \mathbb{R}^6 \rightarrow \mathbb{R}^{64} \quad (\text{same weights as L}) \quad (13)$$

6. Message MLP

$$\mathbf{m}_L = \text{ResMLPBlock}(\mathbf{h}_{\text{edge}}^L) \quad \mathbb{R}^{64} \rightarrow \mathbb{R}^{64} \quad (14)$$

$$\mathbf{m}_R = \text{ResMLPBlock}(\mathbf{h}_{\text{edge}}^R) \quad \mathbb{R}^{64} \rightarrow \mathbb{R}^{64} \quad (\text{same weights as L}) \quad (15)$$

7. Ghost Masking

$$\mathbf{m}_L \leftarrow \mathbb{K}_{\text{has_left}} \cdot \mathbf{m}_L \quad (\text{zeroed if no left neighbor}) \quad (16)$$

$$\mathbf{m}_R \leftarrow \mathbb{K}_{\text{has_right}} \cdot \mathbf{m}_R \quad (\text{zeroed if no right neighbor}) \quad (17)$$

If bead i is the first bead (no left neighbor), $\mathbb{K}_{\text{has_left}} = 0$ and the entire left message becomes $\mathbf{0} \in \mathbb{R}^{64}$. The network does not need to learn this; it is enforced structurally.

8. Node Update MLP

$$\mathbf{c} = [\mathbf{h}_{\text{self}} \parallel \mathbf{m}_L \parallel \mathbf{m}_R] \in \mathbb{R}^{192} \quad (\text{concatenation}) \quad (18)$$

$$\mathbf{h}_{\text{updated}} = \text{ResMLPBlock}(\mathbf{c}) \quad \mathbb{R}^{192} \rightarrow \mathbb{R}^{64} \quad (19)$$

9. Output Head

$$\hat{\mathbf{y}} = W_{\text{out}} \text{LayerNorm}(\mathbf{h}_{\text{updated}}) + \mathbf{b}_{\text{out}} \quad W_{\text{out}} \in \mathbb{R}^{4 \times 64} \quad (20)$$

10. Output Denormalization

$$\mathbf{y} = \hat{\mathbf{y}} \cdot \boldsymbol{\sigma}_y + \boldsymbol{\mu}_y \quad (21)$$

$$\mathbf{y} = [\Delta x, \Delta y, \Delta v_x, \Delta v_y] \in \mathbb{R}^4 \quad (22)$$

11. State Update

$$\mathbf{pos}_{\text{new}} = \mathbf{pos} + [\Delta x, \Delta y] \quad (23)$$

$$\mathbf{vel}_{\text{new}} = \mathbf{vel} + [\Delta v_x, \Delta v_y] \quad (24)$$

Total: 80,324 trainable parameters — Hidden dim $H = 64$ — Activation: GELU — $K = 1$ message passing round